# Application

## Objectifs du TP

- Comprendre comment annuler des modifications précédentes dans Git en utilisant les commandes `git reset` et `git revert`.
- Comprendre les différences entre `git reset` et `git revert`.
- Comprendre l'utilisation de l'option `--hard` et `--soft` avec la commande `git reset`.

## Étape 1 : Création d'un dépôt Git

1. Créez un nouveau répertoire sur votre ordinateur.
2. Ouvrez un terminal et naviguez jusqu'au répertoire que vous venez de créer.
3. Initialisez le dépôt Git en utilisant la commande suivante : `git init`

## Étape 2 : Ajout de commits

1. Créez un nouveau fichier dans le répertoire et ajoutez-y quelques lignes de code.
2. Ajoutez le fichier au dépôt Git en utilisant la commande suivante : `git add <nom du fichier>`
3. Validez les changements en utilisant la commande suivante : `git commit -m "Ajout d'un nouveau fichier"`
4. Répétez les étapes précédentes pour créer deux autres commits.

## Étape 3 : Utilisation de la commande `git reset` pour annuler des commits

1. Exécutez la commande suivante pour afficher l'historique des commits : `git log`
2. Notez le hash des deux derniers commits.
3. Exécutez la commande suivante pour annuler les deux derniers commits en supprimant les changements effectués : `git reset --hard HEAD~2`
4. Exécutez à nouveau la commande `git log` pour vérifier que les deux derniers commits ont été supprimés, ainsi que toutes les modifications apportées dans ces deux commits.

Note: En utilisant l'option `--hard` avec `git reset`, vous supprimez tous les changements effectués dans les commits annulés. Si vous souhaitez conserver ces

changements, vous pouvez utiliser l'option `--soft` qui annule les commits sans supprimer les modifications effectuées. Les changements apparaissent alors comme des modifications non validées dans l'index de travail.

5. Exécutez la commande suivante pour annuler les deux derniers commits en gardant les changements effectués : `git reset --soft HEAD~2`
6. Exécutez la commande `git status` pour vérifier les modifications en attente.
7. Ajoutez les modifications en attente à l'index en utilisant la commande suivante : `git add .`
8. Validez les modifications en utilisant la commande suivante : `git commit -m "Annulation des deux derniers commits"`

# Étape 4 : Utilisation de la commande `git revert` pour annuler des commits

1. Exécutez la commande suivante pour afficher l'historique des commits : `git log`
2. Notez le hash du dernier commit.
3. Exécutez la commande suivante pour annuler le dernier commit en créant un nouveau commit de "revert": `git revert HEAD`

Note: Contrairement à `git reset`, qui supprime des commits et toute leur histoire, la commande `git revert` crée un nouveau commit qui annule les modifications apportées par un ou plusieurs commits. Cela permet de conserver l'historique des modifications et de les annuler de manière sûre, sans risque de perte de données.

4. Git ouvre votre éditeur de texte par défaut pour saisir un message de commit. Le message de commit par défaut est "Revert 'hash du commit': nom du commit", où "nom du commit" est le message de commit original du commit annulé et "hash du commit" est le hash du commit annulé. Vous pouvez modifier le message de commit si vous le souhaitez, puis enregistrez et quittez l'éditeur de texte pour créer le nouveau commit de revert.
5. Exécutez la commande `git log` pour vérifier que le nouveau commit de "revert" a été créé, annulant les modifications apportées par le commit précédent.

Note: La commande `git revert` ne supprime pas les commits précédents, elle ajoute simplement un nouveau commit qui annule les modifications effectuées dans le commit ciblé. Si vous voulez supprimer des commits précédents, vous devez utiliser la commande `git reset`.